

Федеральное государственное автономное образовательное учреждение
высшего образования «Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Лабораторная работа №2

По дисциплине

“Распределенные системы хранения данных”

Вариант: 335200

Выполнил:
Стрельбицкий Илья Павлович

Группа: Р33101

Преподаватель:
Шешуков Дмитрий Михайлович

Санкт-Петербург, 2024г

Текст задания

Цель работы - на выделенном узле создать и сконфигурировать новый кластер БД Postgres, саму БД, табличные пространства и новую роль, а также произвести наполнение базы в соответствии с заданием. Отчёт по работе должен содержать все команды по настройке, скрипты, а также измененные строки конфигурационных файлов.

Способ подключения к узлу из сети Интернет через helios:

```
ssh -J sXXXXXX@helios.cs.ifmo.ru:2222 postgresY@pgZZZ
```

Способ подключения к узлу из сети факультета:

```
ssh postgresY@pgZZZ
```

Номер выделенного узла pgZZZ, а также логин и пароль для подключения Вам выдаст преподаватель.

Этап 1. Инициализация кластера БД

- Директория кластера: \$HOME/tda19
- Кодировка: ISO_8859_5
- Локаль: английская
- Параметры инициализации задать через аргументы команды

Этап 2. Конфигурация и запуск сервера БД

- Способ подключения: сокет TCP/IP, только localhost
- Номер порта: 9200
- Остальные способы подключений запретить.
- Способ аутентификации клиентов: по паролю SHA-256
- Настроить следующие параметры сервера БД:

- max_connections
- shared_buffers
- temp_buffers
- work_mem
- checkpoint_timeout
- effective_cache_size
- fsync
- commit_delay

Параметры должны быть подобраны в соответствии с аппаратной конфигурацией: оперативная память 12ГБ, хранение на жёстком диске (HDD).

- Директория WAL файлов: \$PGDATA/pg_wal
- Формат лог-файлов: .csv
- Уровень сообщений лога: NOTICE
- Дополнительно логировать: попытки подключения и завершение сессий

Этап 3. Дополнительные табличные пространства и наполнение базы

- Создать новые табличные пространства для временных объектов: \$HOME/mto69
- На основе template0 создать новую базу: leftgreenroad
- Создать новую роль, предоставить необходимые права, разрешить подключение к базе.
- От имени новой роли (не администратора) произвести наполнение ВСЕХ созданных баз тестовыми наборами данных. ВСЕ табличные пространства должны использоваться по назначению.
- Вывести список всех табличных пространств кластера и содержащиеся в них объекты.

Ход выполнения

Этап 1. Инициализация кластера БД

Способ подключения к узлу из сети Интернет через helios:

```
ssh s335159@helios.cs.ifmo.ru:2222 postgres8@pg131
```

Способ подключения к узлу из сети факультета:

```
ssh postgres8@pg131
```

Так как кодировка ISO_8859_5 отсутствует на сервере, была выбрана схожая кодировка ISO_8859_15

Команда:

```
initdb --encoding=ISO_8859_15 --locale=en_US.ISO8859-15 --pgdata=$HOME/tda19
```

```
[postgres8@pg131 ~]$ initdb --encoding=ISO_8859_15 --locale=en_US.ISO8859-15 --pgdata=$HOME/tda19
Файлы, относящиеся к этой СУБД, будут принадлежать пользователю "postgres8".
От его имени также будет запускаться процесс сервера.

Кластер баз данных будет инициализирован с локалью "en_US.ISO8859-15".
Выбрана конфигурация текстового поиска по умолчанию "english".

Контроль целостности страниц данных отключён.

создание каталога /var/db/postgres8/tda19... ок
создание подкаталогов... ок
выбирается реализация динамической разделяемой памяти... posix
выбирается значение max_connections по умолчанию... 100
выбирается значение shared_buffers по умолчанию... 128MB
выбирается часовой пояс по умолчанию... W-SU
создание конфигурационных файлов... ок
выполняется подготовительный скрипт... ок
выполняется заключительная инициализация... ок
сохранение данных на диске... ок

initdb: предупреждение: включение метода аутентификации "trust" для локальных подключений
Другой метод можно выбрать, отредактировав pg_hba.conf или используя ключи -A,
--auth-local или --auth-host при следующем выполнении initdb.

Готово. Теперь вы можете запустить сервер баз данных:

    pg_ctl -D /var/db/postgres8/tda19 -l файл_журнала start

[postgres8@pg131 ~]$
```

Этап 2. Конфигурация и запуск сервера БД

Настройка параметров подключения:

Перед тем как настраивать подключение, требуется задать пользователю пароль, так как в противном случае подключится не получится.

Прописываем нужную нам конфигурацию, но пока что вместо метода scram-sha-256 указываем trust

#	TYPE	DATABASE	USER	ADDRESS	METHOD
# rshd lab2					
host	all		all	127.0.0.1/32	trust
# "local" is for Unix domain socket connections only					
local	all		all		reject
# IPv4 local connections:					
host	all		all	127.0.0.1/32	reject
# IPv6 local connections:					
host	all		all	::1/128	reject
# Allow replication connections from localhost, by a user with the					
# replication privilege.					
local	replication		all		reject
host	replication		all	127.0.0.1/32	reject
host	replication		all	::1/128	reject

Затем подключаемся к бд и задаем пользователю пароль.

```
postgres@pg131 ~]$ psql -h 127.0.0.1 -p 9200 postgres postgres8
psql (14.2)
Введите "help", чтобы получить справку.

postgres=#
postgres=# \q
postgres@pg131 ~]$ psql -h 127.0.0.1 -p 9200 postgres postgres8
psql (14.2)
Введите "help", чтобы получить справку.

postgres=# \password postgres8
Введите новый пароль для пользователя "postgres8":
Повторите его:
postgres=#
```

Изменяем метод на scram-sha-256:

```
# TYPE DATABASE USER ADDRESS METHOD
# rshd lab2
host all all 127.0.0.1/32 scram-sha-256
# "local" is for Unix domain socket connections only
local all all reject
# IPv4 local connections:
host all all 127.0.0.1/32 reject
# IPv6 local connections:
host all all ::1/128 reject
# Allow replication connections from localhost, by a user with the
# replication privilege.
local replication all reject
host replication all 127.0.0.1/32 reject
host replication all ::1/128 reject
```

Файл postgresql.conf

Способ подключения к БД — TCP/IP socket, номер порта 9200.

```
port = 9200 # (change requires restart)
```

Устанавливаем порт сервера

Способ аутентификации клиентов — по паролю sha-256

```
# - Authentication -

#authentication_timeout = 1min # 1s-600s
password_encryption = scram-sha-256 # scram-sha-256 or md5
#db_user_namespace = off
```

Директория WAL файлов — \$PGDATA/pg_wal

```
# - Archiving -

archive_mode = on # enables archiving; off, on, or always
# (change requires restart)
archive_command = 'cp %p $PGDATA/pg_wal/' # command to use to archive a log file segment
```

Команда локальной оболочки, выполняемая для архивирования завершенного сегмента файла WAL. Если `archive_mode = off`, тогда `archive_command` — игнорируется. Если `archive_command` является пустой строкой (по умолчанию) при включенном `archive_mode`, архивирование WAL временно отключается, но сервер продолжает накапливать файлы сегмента WAL в ожидании, что вскоре будет предоставлена команда.

Формат лог-файлов — csv

Параметр включает сборщик сообщений (logging collector). Это фоновый процесс, который собирает отправленные в `stderr` сообщения и перенаправляет их в журнальные файлы. Такой подход зачастую более полезен чем запись в `syslog`, поскольку некоторые сообщения в `syslog` могут не попасть.

При включённом `logging_collector`, определяется каталог, в котором создаются журнальные файлы. Можно задавать как абсолютный путь, так и относительный от каталога данных кластера

```
# - Where to Log -

log_destination = 'csvlog'
#log_destination = 'stderr'          # Valid values are combinations of
                                     # stderr, csvlog, syslog, and eventlog,
                                     # depending on platform.  csvlog
                                     # requires logging_collector to be on.

# This is used when logging to stderr:
logging_collector = on               # Enable capturing of stderr and csvlog
                                     # into log files. Required to be on for
                                     # csvlogs.
                                     # (change requires restart)

# These are only used if logging_collector is on:
log_directory = 'log'               # directory where log files are written,
```

Уровень сообщений лога — NOTICE

Управляет минимальным уровнем сообщений, записываемых в журнал сервера. Допустимые значения DEBUG5, DEBUG4, DEBUG3, DEBUG2, DEBUG1, INFO, NOTICE, WARNING, ERROR, LOG, FATAL и PANIC. Каждый из перечисленных уровней включает все идущие после него. Чем дальше в этом списке уровень сообщения, тем меньше сообщений будет записано в журнал сервера. По умолчанию используется WARNING.

```
# - When to Log -

log_min_messages = notice
```

Дополнительно логгировать — попытки подключения и завершение сессий

```
#log_checkpoints = off
log_connections = on
log_disconnections = on
#log_duration = off
#log_error_verbosity = default
```

Настроить следующие параметры сервера БД: max_connections, shared_buffers, temp_buffers, work_mem, checkpoint_timeout, effective_cache_size, fsync, commit_delay

```
max_connections = 200                # (change requires restart)
```

max_connections: 200, выше этого значения нет смысла разгонять - есть смысл вместо увеличения максимального количества соединений использовать пулы.

```
shared_buffers = 3GB                 # min 128kB
```

shared_buffers: увеличил до 3 Гб. Задаёт объём памяти, который будет использовать сервер для буферов в разделяемой памяти. Рекомендуемое начальное значение это 25% от объёма оперативной памяти, оно и будет взято.

```
temp_buffers = 32MB # min 800kB
```

temp_buffers: 32 Мб. Задаёт максимальное число временных буферов для каждого сеанса. Используется только для хранения временных таблиц в памяти.

```
work_mem = 8MB # min 64kB
```

work_mem: 8 Мб. Эта конфигурация используется для сложной сортировки. Если вы собираетесь выполнять сложную сортировку, то идеально увеличить значение этого параметра, чтобы увидеть хорошие результаты.

temp_buffers и work_mem суммарно могут максимально использовать при 200 соединениях 8000 Мб (7,8125 Гб), что в сумме с учетом shared_buffers занимает 11072 Мб (10,8125 Гб).

```
checkpoint_timeout = 5min # range 30s-1d
```

checkpoint_timeout: 5 min (дефолтное значение). Уменьшение параметра приводит к учащению контрольных точек и повышению нагрузки, увеличение параметра приводит к большему времени, которое потребуется для восстановления после сбоя.

```
effective_cache_size = 4GB
```

effective_cache_size: 4 Гб (дефолтное значение). Определяет представление планировщика об эффективном размере дискового кеша, доступном для одного запроса. Значение должно быть больше или равно shared_buffers, поэтому оставляю значение по умолчанию в 4 Гб.

```
fsync = on # flush data to disk for crash safety
```

fsync = on: (дефолтное значение) — выключение параметра приводит к росту производительности, но при этом появляется значительный риск потери всех данных при внезапном выключении питания.

```
commit_delay = 0 # range 0-100000, in microseconds
```

commit_delay: 0. Добавляет задержку перед началом очистки WAL. При асинхронной обработке коммитов оптимальным значением является 0.

Так как хранение происходит на жестком диске, то для того чтобы ускорить обслуживание транзакций, сделаем асинхронную запись журнала. При синхронной записи продолжение работы невозможно, пока журнальные записи не окажутся на диске, так как HDD достаточно медленный, по этой причине есть смысл использовать асинхронную запись журнала, при которой не требуется для продолжения работы запись журнала на диск.

```
synchronous_commit = off
```

Получаем следующие значения:

```
max_connections = 200
```

```
shared_buffers = 3 Гб
```

```
temp_buffers = 32 Мб
```

```
work_mem = 8 Мб
```

```
checkpoint_timeout = 5 min
```

```
effective_cache_size = 4 Гб
```

```
fsync = on
```

```
commit_delay = 0
```

Этап 3. Дополнительные табличные пространства и наполнение базы

Создать новые табличные пространства для временных объектов \$HOME/mto69

```
[postgres8@pg131 ~]$ mkdir mto69
[postgres8@pg131 ~]$ ls
mto69  tda19
```

Создаем директорию.

```
[postgres8@pg131 ~]$ psql -h 127.0.0.1 -p 9200 postgres postgres8
Пароль пользователя postgres8:
psql (14.2)
Введите "help", чтобы получить справку.

postgres=# CREATE TABLESPACE temp_tablespaces LOCATION '/var/db/postgres8/mto69';
CREATE TABLESPACE
```

Подключаемся к бд и добавляем табличное пространство

На основе template0 создать новую базу — leftgreenroad

```
postgres=# CREATE DATABASE leftgreenroad WITH TEMPLATE = template0;
CREATE DATABASE
```

Создать новую роль, предоставить необходимые права, разрешить подключение к базе.

```
postgres=# CREATE USER user1 WITH PASSWORD 'user1';
CREATE ROLE
postgres=# GRANT TEMPORARY, CONNECT ON DATABASE leftgreenroad TO user1;
GRANT
```

```
CREATE USER user1 WITH PASSWORD 'user1';
GRANT TEMPORARY, CONNECT ON DATABASE leftgreenroad TO user1;
```

От имени новой роли (не администратора) произвести наполнение BCEX созданных баз тестовыми наборами данных. BCE табличные пространства должны использоваться по назначению.

```
CREATE TABLE IF NOT EXISTS first (
    id serial PRIMARY KEY,
    column1 int,
    column2 int,
    column3 int
);
CREATE TABLE IF NOT EXISTS second (
    id serial PRIMARY KEY,
    column1 int,
    column2 int,
    column3 int
);
CREATE TABLE IF NOT EXISTS third (
    id serial PRIMARY KEY,
    column1 int,
    column2 int,
```



```
column3 int  
);
```

```
INSERT INTO first (id, column1, column2, column3) VALUES (default, 0, 1, 2);  
INSERT INTO second (id, column1, column2, column3) VALUES (default, 0, 1, 2);  
INSERT INTO third (id, column1, column2, column3) VALUES (default, 0, 1, 2);
```

```
leftgreenroad=> DROP TABLE first, second, third;  
DROP TABLE  
leftgreenroad=> CREATE TABLE IF NOT EXISTS first (  
    id serial,  
    column1 int,  
    column2 int,  
    column3 int  
);  
CREATE TABLE IF NOT EXISTS second (  
    id serial,  
    column1 int,  
    column2 int,  
    column3 int  
);  
CREATE TABLE IF NOT EXISTS third (  
    id serial,  
    column1 int,  
    column2 int,  
    column3 int  
);  
  
INSERT INTO first (id, column1, column2, column3) VALUES (default, 0, 1, 2);  
INSERT INTO second (id, column1, column2, column3) VALUES (default, 0, 1, 2);  
INSERT INTO third (id, column1, column2, column3) VALUES (default, 0, 1, 2);  
CREATE TABLE  
CREATE TABLE  
CREATE TABLE  
INSERT 0 1  
INSERT 0 1  
INSERT 0 1
```

Вывести список всех табличных пространств кластера и содержащиеся в них объекты.

```
leftgreenroad=> SELECT spcname FROM pg_tablespace;  
    spcname  
-----  
pg_default  
pg_global  
temp_tablespaces  
(3 строки)
```

Вывод всех табличных пространств кластера.

```
-----  
pg_toast_1262  
pg_toast_1262_index  
pg_toast_2964  
pg_toast_2964_index  
pg_toast_1213  
pg_toast_1213_index  
pg_toast_1260  
pg_toast_1260_index  
pg_toast_2396  
pg_toast_2396_index  
pg_toast_6000  
pg_toast_6000_index  
pg_toast_3592  
pg_toast_3592_index  
pg_toast_6100  
pg_toast_6100_index  
pg_database_datname_index  
pg_database_oid_index  
pg_db_role_setting_databaseid_rol_index  
pg_tablespace_oid_index  
pg_tablespace_spcname_index  
pg_authid_rolname_index  
pg_authid_oid_index  
pg_auth_members_role_member_index  
pg_auth_members_member_role_index  
pg_shdepend_depender_index  
pg_shdepend_reference_index  
pg_shdescription_o_c_index  
pg_replication_origin_roident_index  
pg_replication_origin_roname_index  
pg_shseclabel_object_index  
pg_subscription_oid_index  
pg_subscription_subname_index  
pg_authid  
pg_subscription  
pg_database  
pg_db_role_setting  
pg_tablespace  
pg_auth_members  
pg_shdepend  
pg_shdescription  
pg_replication_origin  
pg_shseclabel  
(43 строки)
```

```
SELECT relname FROM pg_class WHERE reltablespace IN (SELECT oid FROM pg_tablespace);
```

Вывод содержащихся объектов в табличных пространствах данных

Вывод

В ходе выполнения данной лабораторной работы были изучены способы создания и настройки кластера баз данных PostgreSQL, и также работа с табличными пространствами кластера.